Table of contents

Basic information

Encodings

Data types

Indexes

Show all...

BrightScript Profiler file format specification

Developers can follow this specification to build a custom profiling tool that analyzes app performance metrics such as CPU and memory usage. This specification describes the entries in the header, body, and footer of a BrightScript profiler file (.bsprof) file.

Basic information

The following section details the encoding, data types, and indexes used in the BrightScript file format.

Encodings

Encoding	Definition
float32le	IEEE-754 32-bit floating point value in little-endian order (https://en.wikipedia.org/wiki/Single-precision_floating-point_format)
uint8	Unsigned 8-bit value (unsigned char and octet)
uint8[]	Sequence of uint8 values of defined length
utf8z	UTF-8 character sequence (terminated with a zero byte)

Encoding	Definition
varint	Variable-length integer (https://tools.ietf.org/html/draft-rfernando-protocol-buffers-00)

Data types

The fields in the file format may be one of the following data types, which can hold any encoded value for a field:

Data type	Definition
float	32-bit (or greater) floating point value
string	In-memory string object (this may be any string type that will hold unicode characters).
bool	uint8, where 0 is false and any other value is true
uint8	Unsigned 8-bit value (unsigned char and octet)
uint8[]	Array of uint8 values
uint32	Unsigned 32-bit integer
uint64	Unsigned 64-bit integer
int64	Signed 64-bit integer

Indexes

All indexes are 1-based (rather than 0-based). A value of 0 indicates a null or invalid ID.

Data structure

BrightScript profiling data is provided in a streaming format that contains a series of record types that can appear anywhere in the file. This allows the streaming of data to the network while the app runs. Overall, the file format incudes the following:

- header
- body (the body contains the stream of data entries)
- end of entries marker

- footer

Header

The header of the BrightScript profiling file includes the following fields:

Name	Encoding	Type	Description
magic	uint8[8]	uint64	['b','s','p','r','o','f','\0','\0'] = [0x62,0x73,0x70,0x72,0x6f,0x66,0x00,0x00] = 0x666f72707362LLU (little-endian)
major version	varint	uint32	Major version of this file format specification. A change in the major version implies incompatibility with previous formats.
minor version	varint	uint32	Minor version of this file format specification. A change in the minor version is compatible with tools that read the same major version.
version patch level	varint	uint32	Patch level of this version, which is effectively a "tertiary version" (for example, the patch level for "1.2.3" would be "3". A change in the patch level is compatible with tools that support the same major version.
header size	varint	uint32	The size of this header, in bytes from the beginning of the file. Tools that read this format should read all of the header fields it understands, and then skip to the body of the data, below.
requested sample ratio	float32ie	float32	Sample ratio requested by the operator
actual sample ratio	float32ie	float32	Actual sample ratio performed by the profiler. This may differ from the requested ratio due to other profiler options or platform limitations.
line-specific data	varint	bool	Indicates whether CPU, memory, and path element entries include line offset values used to reference the entry to a specific line of source code.
memory operations	varint	bool	Indicates whether each call executive module path element includes a list of memory operations performed at that location in the app's source code.
timestamp of target run start	varint	uint64	Timestamp when the target program was started since 1970-01-01T00:00:00.000Z (in milliseconds).

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Name	Encoding	Type	Description
target name	utf8z	string	Human-readable name of profiling target (on Roku devices, this is the app title).
supplemental information	utf8z	string	Human-readable string with additional information about the profiling target or the device. This field is always present, but it may be an empty string ("").
target Version	utf8z	string	Human-readable version of the profiling target (on Roku devices, this is the app version in the manifest).
device vendor name	utf8z	string	Human-readable name of the device vendor.
device model number	utf8z	string	Human-readable model number of the device.
device firmware version	utf8z	string	Human-readable version of the firmware on the device.
padding	uint8[]	uint8[]	The padding with zeroes written by the profiler for implementation-specific reasons. Reading the header size bytes from the start of the file skips this.

Body

The body of the BrightScript profiling file contains a stream of entries of varying types. If an entry type depends on one or more other entries, the dependent entry is listed after the required entries. For example, each CPU entry requires at least three total entries (listed in order):

- One or more string table entries.
- One or more path element entries.
    - Each path element references one level in a call chain; therefore, multiple path elements are needed to define a full call path.
    - Each path element references a function name and file name using a string ID, which is defined in a string table entry.
- The actual CPU entry, which references a path element entry where the measurement occurred.

Most entries are referenced multiple times, but they are not duplicates. For example, in the previous CPU entry example, the string table and path element entries would be referenced many times for other CPU or memory operation entries.

Common entry format

Each entry is a stream of bytes, beginning with a varint-encoded unsigned 64-bit (uint64) entry tag, where the least significant three bits define the entry type. Once this entry is parsed, no value in the entry tag payload is greater than 32 bits.

Entry tag (variant-encoded uint64)		Following bytes
Bits 63..3 (most significant 61 bits) This is the "entry tag payload". The meaning of this payload is determined by entry tag type.	Bits 2..0 (least significant 3 bits) This is the entry tag type.	Multiple bytes, determined by entry tag type.

String table entry

A string table entry is a stream of bytes that defines a single string in the global string table, which associates one unsigned integer String ID (strid) to a unicode string. The String ID is a uint32 value, where 0 is explicitly invalid and is used to mark a null value.

String IDs are stored in several fields within the profiling data. The indexes are 1-based, where 0 represents a nonexistent or NULL string.

Entry tag (variant-encoded uint64)		String
Bits 34..3 string ID (uint32)	Bits 2..0 (least significant 3 bits) Literal 0x0	utf8z string

Executable module entry

Each executable module is a stream of bytes that represents a block of code that runs independently of other modules.

The modules listed in a profiler file are run simultaneously within a single profiler target. For example, each SceneGraph component is represented by a separate executable module.

Entry Tag (variant-encoded uint64)		Thread name
Bits 34..3 module ID (uint32)	Bits 2..0 (least significant bits) Literal 0x1	varint-encoded String ID (strid)

Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

Path element entry

A path element represents a single entry in a call path that is typically a function name. The path element ID is a uint32, where 0 is explicitly invalid and is used to mark a null value.

Root

Entry tag (varint-encoded uint64)		Calling path element ID	Executive module ID	File name
Bits 34..3 The path element ID (uint32).	Bits 0..2 (least significant 3 bits) Literal 0x2	varint-encoded literal 0 value. This null calling path element ID specifies that this is a root entry for its executive module.	varint-encoded uint32 The module ID for which this element is a root entry.	varint-encoded String ID The source file where this func was defined.

Non-root (chained)

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Entry tag (varint-encoded uint64)		Calling path element ID	Line offset in caller	File name
Bits 34..3 The path element ID (uint32).	Bits 0..2 (least significant 3 bits) Literal 0x2	varint-encoded path element ID of caller	varint-encoded uint32 A 1-based offset of the calling line of code, into the function at the end of the calling path. To calculate the actual line number in the source file, a custom tool should use the following formula : pathEntry.lineNumber + memoryEntry.lineOffset - 1. This value is only present if the file header specifies that line data is included.	varint-encoded String ID The source file where this function was defined.

Memory operation entry

A memory operation is a stream of bytes.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Entry tag (varint-encoded uint64)			Line offset
Bits 36..5 The path element ID (the uint32) that generated this operation).	Bits 4..3 Operation type, which may be on the following values: • 0: alloc • 1: free • 2: free_realloc (a free_realloc operation is a free operation that occurs as part of a realloc, and should be immediately followed by an alloc operation).	Bits 2..0 (least significant 3 bits) Literal 0x3	varint-encoded uint32 A 1-based offset of the line of code, into the function at the end of the call path. To calculate the actual line number in the source file, a custom tool should use the following formula : pathEntry.lineNumber + memoryEntry.lineOffset - 1. This value is only present if the file header specifies that line data is included.

CPU measurement entry

A CPU measurement entry is a stream of bytes. The custom tool should treat CPU entries as incremental values, and it should sum all CPU entries for a given call path (for devices Roku OS 9.0 or lower, only one CPU entry is generated for each unique call path).

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Entry tag (varint-encoded uint64)		Line offset	CPU self
Bits 34..3 The path element ID (the uint32) that generated this operation).	Bits 2..0 (least significant 3 bits) Literal 0x4	varint-encoded uint32 A 1-based offset of the line of code, into the function at the end of the call path. To calculate the actual line number in the source file, a custom tool should use the following formula : pathEntry.lineNumber + memoryEntry.lineOffset - 1. This value is only present if the file header specifies that line data is included.	varint-encoded uint32 The incremental CPU time spent on the call path.

Path call count entry

A count of calls made into a specific call path. The custom tool should treat each call count entry as an incremental value. One or more call count entries may appear for each unique call path (for devices Roku OS 9.0 or lower, only one call count entry is generated for each unique call path).

Entry tag (varint-encoded uint64)		Call count
Bits 34..3 The path element ID (uint32) that was called.	Bits 2..0 (least significant 3 bits) Literal 0x5	Varint-encoded uint32 The number of times a function was called on the specified call path (incremental value).

End of entries marker

The end of the entries in the data stream is marked with an End Of Entries tag.

Entry tag (varint-encoded uint64)	
Literal 0x0	

File footer

The file footer is a list of fields that is similar to the file header.

Name	Encoding	Type	Description
Timestamp of target run end	varint	uint64	Timestamp when the target program ended since 1970-01-01T00:00:00.000Z (in milliseconds).

Previous Next

Terms and Conditions

B2B Privacy Notice (https://docs.roku.com/published/b2bprivacypolicy/en/us)

Developer Site (https://developer.roku.com)

About ads & cookies (https://docs.roku.com/published/cookiepolicy)

My Privacy Choices
(https://privacy.roku.com/ccpa)
Developer Terms & Agreements (https://developer.roku.com/docs/features/legal/developer-terms.md)

CA B2B Privacy Notice

© 2008-2026 Roku, Inc. (https://www.roku.com)